

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	P. Ahlen Abishek	Reg.No.:	192311034
Department:	CSE - dept	Date:	26-07-26

ANNEXURE A

Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μ s
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct:

I P. Allen Abishek (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

A
Signature of the student:

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1	2	2	2	1	1
Q2	2	2	2	1	1
Q3	2.5	2.5	1.5	1.5	0.5
Q4	1.5	1.5	1.5	1.5	0.5
Q5	2	2	2	1	0.5

RUBRICS FOR CALCULATION:

38.5

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25% 2.5	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25% 2.5	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25% 2.5	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15% 2.5	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10% 1	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

SIMATS ENGINEERING

Department of Computer Science & Engineering

Course Code: CSA12 | Course Title: Computer Architecture | CO Assessed: CO5

ASSESSMENT TOOL 1 – CONSTRAINT-BASED PROBLEM SOLVING

Student Name: Arena Abishek

Reg. No.: 192311034

Department: Computer Science & Engineering

Date: _____

Q1. Real-Time Embedded System (CPU < 15%, sensors interrupt every 1 ms, vectored interrupts, MMIO vs I/O-mapped)

Given Constraints:

- CPU utilization must stay below 15%
- Multiple sensors generate interrupts every 1 ms
- Must use vectored interrupt handling
- Must choose between memory-mapped and I/O-mapped I/O for optimal latency

Problem Understanding:

The system must service frequent, high-rate sensor events without saturating the CPU, and each sensor's ISR must be reached directly through its own vector rather than through a slow polled dispatch loop.

Constraint Analysis (priority order):

1. CPU budget (<15%) is the hardest constraint — it rules out polling, which would consume CPU continuously at a 1 ms sensor rate.
2. Interrupt frequency (1 kHz aggregate) means ISR execution time must be very short, or interrupts will queue up and violate the CPU budget indirectly.
3. Latency-optimal addressing is secondary — chosen once the interrupt mechanism is fixed.

Solution Development:

Application of Concepts:

Vectored interrupts give $O(1)$ dispatch to the correct ISR (no software polling loop scanning devices), which is what keeps CPU usage low at a 1 kHz interrupt rate. I/O-mapped access is preferred for sensor status/data registers when it measures lower latency, since it avoids contention with the shared memory bus that other tasks are using.

Justification:

Vectored interrupts are chosen over polling specifically because polling would require the CPU to continuously check sensor status, violating the <15% utilization limit; the direct hardware vector lookup replaces this with near-zero idle overhead. I/O-mapped I/O is justified when it benchmarks faster for small, frequent register reads, which matches the sensor access pattern better than memory-mapped I/O's larger address-space overhead.

Q2. High-Speed NVMe/PCIe Storage (>3 GB/s, no CPU bulk transfer, DMA + interrupt)

Given Constraints:

- Throughput must exceed 3 GB/s
- Must use NVMe over PCIe
- CPU must not handle bulk data transfer
- DMA controller must integrate with interrupt notification

Problem Understanding:

The storage subsystem must sustain very high throughput while keeping the CPU almost entirely out of the data path — its only role is to set up the transfer and respond to a completion interrupt.

Constraint Analysis (priority order):

1. Throughput (>3 GB/s) dictates the bus and protocol choice — only PCIe (with enough lanes) and NVMe's parallel queue model can sustain this, ruling out legacy interfaces like SATA/AHCI.
2. Zero CPU bulk-transfer involvement is the defining architectural constraint — it forces a DMA-based design rather than programmed I/O.
3. Interrupt integration is needed so the CPU learns of completion without polling, which would itself consume CPU cycles.

Solution Development:

Application of Concepts:

NVMe's multiple parallel submission/completion queues over PCIe are used specifically to reach the >3 GB/s throughput target, since a single legacy queue would bottleneck well below that. DMA moves all bulk data directly between the SSD controller and memory, satisfying the 'no CPU bulk transfer' constraint.

Justification:

PCIe/NVMe is justified over SATA/AHCI because it offers many more lanes and parallel queues, which is required to physically sustain >3 GB/s. DMA with interrupt-on-completion is justified because it is the only combination that removes the CPU from the transfer path while still telling the CPU exactly when the data is ready.

Q3. Multi-Device Bus Architecture (6 USB + 2 Thunderbolt, no starvation, HW+SW interrupts, fair arbitration)

Given Constraints:

- 6 peripherals on USB, 2 high-speed devices on Thunderbolt
- Must avoid interrupt starvation
- Must use both hardware and software interrupts
- Must ensure fair bus arbitration

Problem Understanding:

Two bus classes with very different speeds share the system; USB devices must not be starved by the higher-priority, higher-bandwidth Thunderbolt devices, and both hardware-triggered and software-triggered events must be supported.

Constraint Analysis (priority order):

1. Starvation avoidance is the primary constraint — a naive fixed-priority scheme favouring Thunderbolt would let 6 USB devices wait indefinitely, so this must be designed for first.
2. Fair arbitration follows directly — it is the mechanism that prevents starvation while still respecting Thunderbolt's higher throughput class.
3. Hardware vs software interrupt separation is a supporting requirement — hardware interrupts flag physical device events, software interrupts flag internally raised conditions (e.g., driver-initiated resets).

Solution Development:

Application of Concepts:

Aging-based priority (priority grows with wait time) is applied so that Thunderbolt's naturally higher base priority cannot permanently block the 6 USB devices, directly addressing the starvation constraint. Hardware interrupts handle physical device signalling; software interrupts let the driver stack raise equivalent events (e.g., a forced reset) through the same dispatch path.

Justification:

Aging is justified as the standard, well-understood technique for bounding worst-case wait time in priority schedulers, which is exactly what 'avoid starvation' requires. Using one unified ISR dispatch path for both hardware and software interrupts is justified because it keeps the priority and fairness logic consistent regardless of an event's origin.

Q4. Data Acquisition System (500 MB/s continuous stream, minimal CPU, DMA vs interrupt-driven, MMIO registers)

Given Constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Must use memory-mapped I/O for device registers

Problem Understanding:

A continuous, high-rate data stream must be captured with the least possible CPU involvement, using memory-mapped registers to configure and monitor the acquisition hardware.

Constraint Analysis (priority order):

1. Sustained throughput (500 MB/s) is the dominant constraint — at this rate, per-word interrupt-driven I/O would generate an unmanageable interrupt rate.
2. Minimal CPU intervention reinforces that bulk data movement must bypass the CPU entirely.
3. MMIO for registers is a fixed design choice for configuration/status access, independent of how the bulk data itself moves.

Solution Development:

DMA transfer (chosen):

The DMA controller streams data from the acquisition device directly into a ring buffer in memory using double-buffering; the CPU is interrupted only once per buffer-full event (not per word), giving very low CPU overhead at 500 MB/s.

Interrupt-driven I/O (compared):

Each data unit would raise an interrupt for the CPU to read and store; at 500 MB/s this would translate into an extremely high interrupt rate, dominating the CPU with context-switch overhead and failing the 'minimal CPU intervention' constraint.

```
ALGORITHM DAQ_Stream_Capture(device, ring_buffer)
BEGIN
    // Registers configured via MMIO
    MEM[device.CONTROL_REG] = START_ACQUISITION
    MEM[device.RATE_REG]    = TARGET_RATE

    FOR EACH half OF ring_buffer DO
        DMA_CONTROLLER.set_destination(half)
        DMA_CONTROLLER.set_length(HALF_BUFFER_SIZE)
    END FOR
    DMA_CONTROLLER.START(CIRCULAR_MODE)    // continuous double-buffered transfer

    CPU.continue_other_tasks()
END

INTERRUPT_HANDLER Buffer_Half_Full_ISR()
BEGIN
    SAVE_CPU_CONTEXT()
    full_half = DMA_CONTROLLER.completed_half()
    PROCESS_OR_OFFLOAD(full_half)    // CPU only touches data once per half-buffer
    RESTORE_CPU_CONTEXT()
    RETURN_FROM_INTERRUPT()
END
```

Application of Concepts:

MMIO maps the acquisition device's control/status registers into normal memory address space, so standard load/store instructions configure it; the actual data path uses DMA with a circular double buffer so the CPU is interrupted only per buffer, not per sample.

Justification:

DMA is justified over interrupt-driven I/O purely on rate: 500 MB/s at, say, 4-byte words would mean well over 100 million interrupts per second, which is architecturally infeasible, whereas DMA reduces this to a handful of buffer-boundary interrupts per second. MMIO is justified for registers because it lets the same instruction set and compiler-generated code handle configuration, simplifying the driver.

Q5. Interrupt Mechanism (nested interrupts, latency < 10 μ s, vectored for high priority, software for background)

Given Constraints:

- Must support nested interrupts
- Maximum interrupt latency must be under 10 μ s
- Must use vectored interrupts for high-priority devices
- Must use software interrupts for background tasks

Problem Understanding:

The interrupt controller must allow a high-priority interrupt to preempt a lower-priority one currently in service (nesting), while guaranteeing that the highest-priority event is always serviced within 10 μ s, and background work is deferred via software interrupts rather than blocking hardware ISRs.

Constraint Analysis (priority order):

1. The 10 μ s latency ceiling is the hardest real-time constraint — it bounds how much work any ISR (or the dispatch path itself) may do before reaching the requesting device's handler.
2. Nesting support is required to meet that latency for high-priority devices even when a lower-priority ISR is already executing.
3. Software interrupts for background tasks are a lower urgency, used to keep non-critical work out of the hard-real-time path entirely.

Solution Development:**Application of Concepts:**

A priority stack lets a higher-priority vectored interrupt immediately preempt a running lower-priority ISR (nesting), while minimal-context saving on entry keeps dispatch overhead well under 10 μ s. Background, non-time-critical work is moved out of the hardware ISR path entirely by raising a low-priority software interrupt that only runs when no hardware interrupt is pending.

Justification:

Vectored dispatch with priority-based preemption is justified because it is the only mechanism that can guarantee a high-priority device's ISR starts almost immediately regardless of what else is executing, which is what the $<10 \mu$ s bound demands. Software interrupts are justified for background tasks because folding them into the same hardware ISR path would risk lengthening hardware interrupt latency, directly threatening the timing constraint.

Certification: I, Arena Abishek, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Signature of the student: _____